

TaskForge — Base de données

MCD · MLD · MPD — méthode Merise — PostgreSQL — v3, 31/07/2026

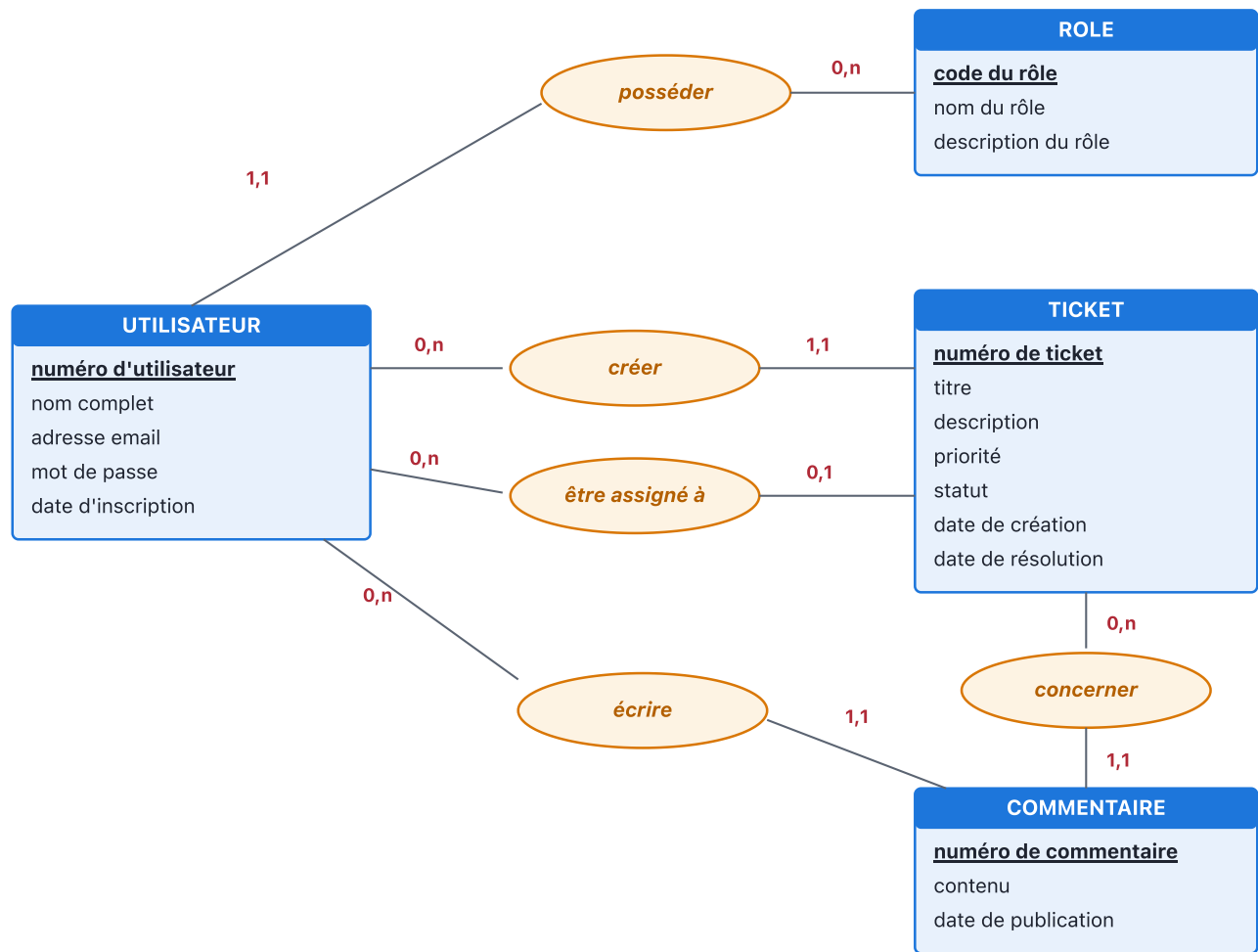
Ce document décrit la base de données de TaskForge, l'application de gestion de tickets du hackathon ForgeWorks. Il suit les trois niveaux de la méthode Merise : le MCD présente les données métier, le MLD les organise en relations, le MPD les implémente dans PostgreSQL.

Règles de gestion

- **RG1** — Un utilisateur a un seul rôle (utilisateur, technicien ou admin). Un rôle peut concerner plusieurs utilisateurs.
- **RG2** — Un ticket est créé par un seul utilisateur. Un utilisateur peut créer plusieurs tickets.
- **RG3** — Un ticket est assigné à au plus un technicien. Un technicien peut traiter plusieurs tickets.
- **RG4** — Un ticket suit le cycle : ouvert → en cours → résolu → fermé. La date de résolution est renseignée au passage en « résolu ».
- **RG5** — Un commentaire a un seul auteur et concerne un seul ticket.
- **RG6** — L'email d'un utilisateur est unique. Son mot de passe est stocké hashé, jamais en clair.

MCD Modèle conceptuel

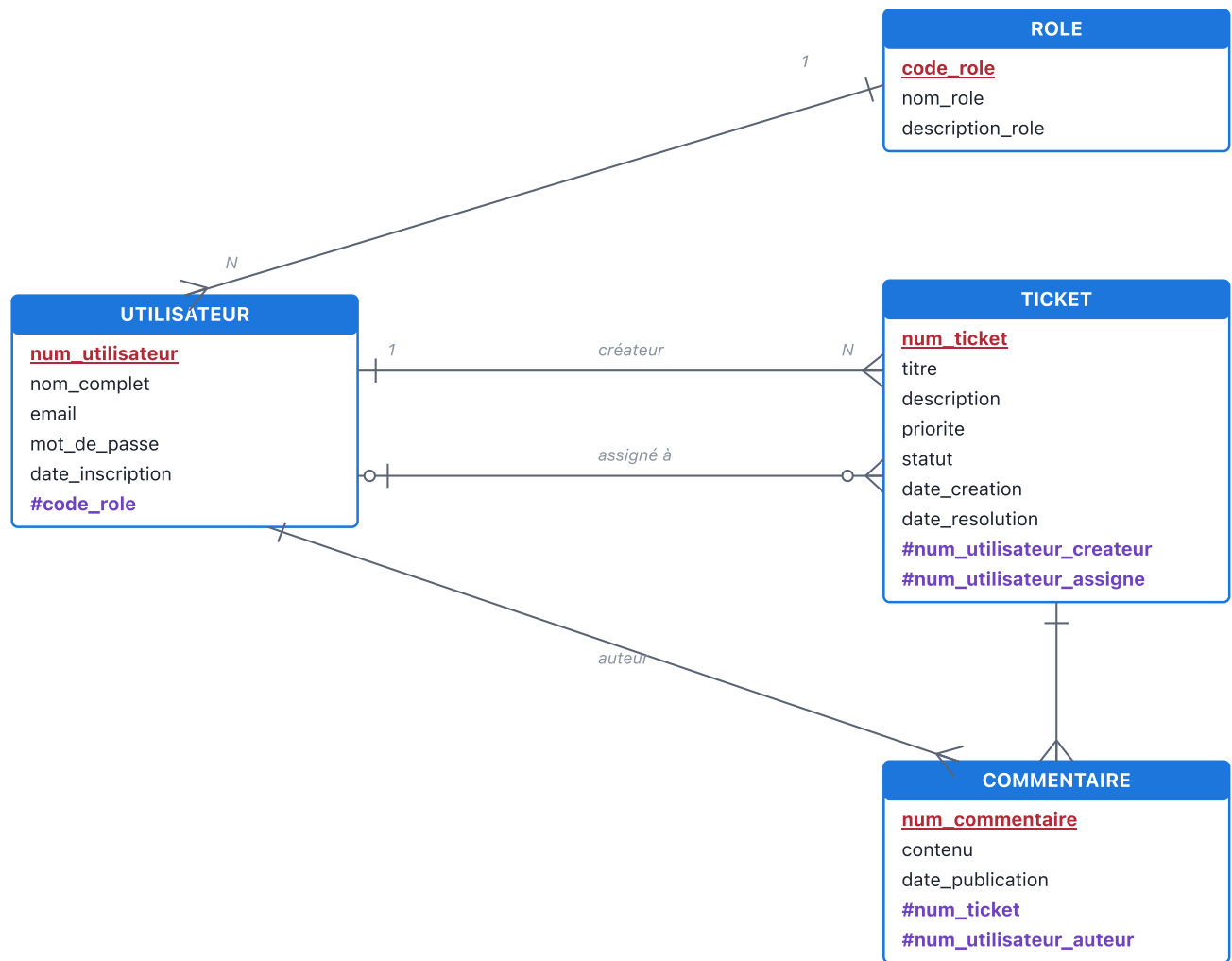
Les données vues par le métier : propriétés en clair, identifiants soulignés, cardinalités (min,max) sur les deux pattes.



Lecture : un utilisateur possède 1,1 rôle — un rôle est possédé par 0,n utilisateurs. Un ticket est créé par 1,1 utilisateur et assigné à 0,1 technicien.

MLD Modèle logique

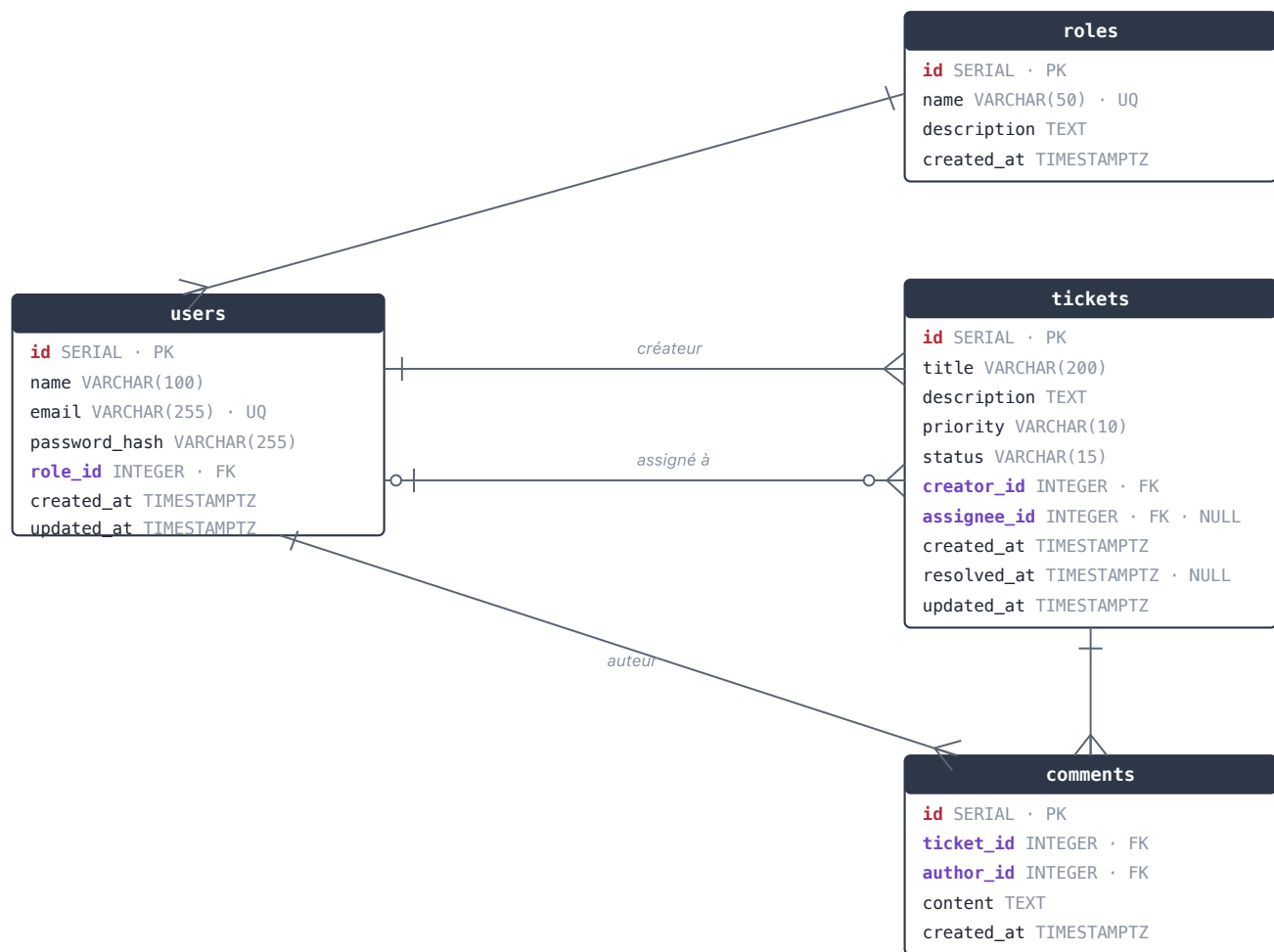
Traduction en relations : clés primaires soulignées, clés étrangères préfixées #, indépendant du SGBD.



Patte d'oie côté N, barre côté 1, cercle quand la participation est optionnelle (un ticket peut n'avoir aucun assigné).

MPD Modèle physique — PostgreSQL

Implémentation concrète : types PostgreSQL, contraintes, index et script de création.



PK en rouge, FK en violet, NULL pour les colonnes optionnelles.

Script de création

```

CREATE TABLE roles (
  id          SERIAL          PRIMARY KEY,
  name        VARCHAR(50)    NOT NULL UNIQUE,
  description  TEXT,
  created_at  TIMESTAMPTZ    NOT NULL DEFAULT now()
);

INSERT INTO roles (name, description) VALUES
  ('utilisateur', 'Crée des tickets et suit leur avancement'),
  ('technicien', 'Résout les tickets qui lui sont assignés'),
  ('admin',      'Gère tout : utilisateurs, tickets, configuration');
  
```

```

CREATE TABLE users (
    id            SERIAL            PRIMARY KEY,
    name          VARCHAR(100) NOT NULL,
    email         VARCHAR(255) NOT NULL UNIQUE,
    password_hash VARCHAR(255) NOT NULL,
    role_id       INTEGER          NOT NULL REFERENCES roles(id),
    created_at    TIMESTAMPTZ      NOT NULL DEFAULT now(),
    updated_at    TIMESTAMPTZ
);

CREATE TABLE tickets (
    id            SERIAL            PRIMARY KEY,
    title         VARCHAR(200) NOT NULL,
    description    TEXT             NOT NULL,
    priority      VARCHAR(10) NOT NULL CHECK (priority IN
        ('basse', 'moyenne', 'haute', 'critique')),
    status        VARCHAR(15) NOT NULL DEFAULT 'ouvert' CHECK (status IN
        ('ouvert', 'en_cours', 'resolu', 'ferme')),
    creator_id    INTEGER          NOT NULL REFERENCES users(id),
    assignee_id   INTEGER          REFERENCES users(id) ON DELETE SET NULL,
    created_at    TIMESTAMPTZ      NOT NULL DEFAULT now(),
    resolved_at   TIMESTAMPTZ,
    updated_at    TIMESTAMPTZ
);

CREATE TABLE comments (
    id            SERIAL            PRIMARY KEY,
    ticket_id     INTEGER          NOT NULL REFERENCES tickets(id) ON DELETE CASCADE,
    author_id     INTEGER          NOT NULL REFERENCES users(id),
    content       TEXT             NOT NULL,
    created_at    TIMESTAMPTZ      NOT NULL DEFAULT now()
);

-- Index pour les filtres, le tri et la recherche (§3 du cahier des charges)
CREATE INDEX idx_tickets_status    ON tickets (status);
CREATE INDEX idx_tickets_priority  ON tickets (priority);
CREATE INDEX idx_tickets_assignee  ON tickets (assignee_id);
CREATE INDEX idx_tickets_created   ON tickets (created_at DESC);
CREATE INDEX idx_comments_ticket   ON comments (ticket_id);

CREATE EXTENSION IF NOT EXISTS pg_trgm;
CREATE INDEX idx_tickets_search ON tickets
    USING gin ((title || ' ' || description) gin_trgm_ops);

-- Mise à jour automatique de updated_at
CREATE OR REPLACE FUNCTION set_updated_at() RETURNS TRIGGER AS $$

```

```
BEGIN
    NEW.updated_at := now();
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_users_updated BEFORE UPDATE ON users
    FOR EACH ROW EXECUTE FUNCTION set_updated_at();
CREATE TRIGGER trg_tickets_updated BEFORE UPDATE ON tickets
    FOR EACH ROW EXECUTE FUNCTION set_updated_at();
```